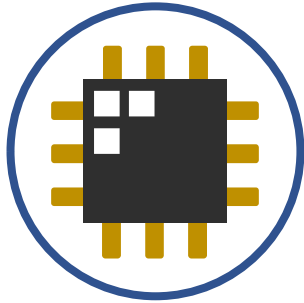


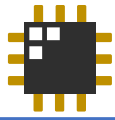


EEE 143: Switching Theory and Digital Logic Design

Lecture 5 – Combinational Logic Blocks Binary Arithmetic

1

Last week, we explored Karnaugh Maps (K-maps), which provide a systematic way to simplify Boolean expressions. The key to mastering K-maps is recognizing patterns, and the more you practice, the easier and quicker it becomes to spot these patterns.



Moving towards higher abstractions

- The previous lectures focused on developing skills for **gate-level** design
- Today, we're moving towards larger and more complex digital blocks
- Having understood binary and logic gates, let's increase complexity a bit by **understanding how computers perform basic arithmetic**

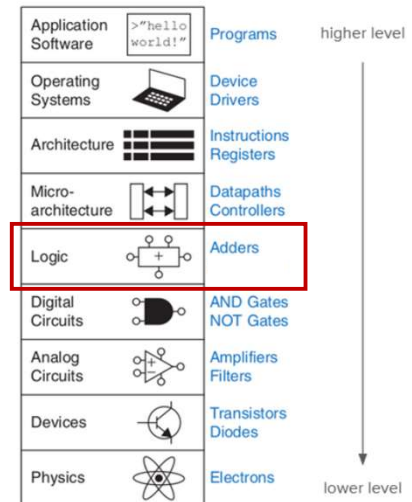
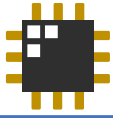


Image credit: *Digital Design and Computer Architecture* (2nd ed.) by Harris, H., Harris, S. (2013).

Essentially, K-maps help us derive a simplified expression from a given truth table, which falls under what we call **gate-level design**—where we manipulate individual logic gates like AND, OR, and NOT.

While gate-level design is fundamental, it becomes impractical for designing more complex systems. If we were to rely solely on K-maps for large-scale digital circuits, the process would be tedious. That's why, as digital systems grow in complexity, we shift our focus from individual logic gates to higher-level functional blocks that perform specific operations.

Starting with this lecture, we'll be diving **into combinational logic blocks**, and a great entry point is arithmetic operations. We'll focus on **addition and subtraction**, but these principles extend naturally to other operations such as multiplication, for instance, is just repeated addition, and division is repeated subtraction. Ultimately, the key takeaway is understanding how computers perform arithmetic at the digital level, which, of course, happens in **binary**.



Binary addition

- Addition in binary is simple!

- $0 + 0 = 0$

- $0 + 1 = 1$

- $1 + 0 = 1$

- $1 + 1 = 10$ (0 carry 1) Note: $1 + 1 = 2$, but $2 = 10_2$

The '+' symbol here means add, not the OR logic

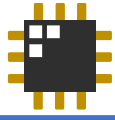
- Similar to decimal, we will end up with a carry when we start exceeding the value that can be placed in a single digit!

3

Addition in binary is very similar to decimal. However, we must constrain ourselves to only two digits, 0 and 1. So $0 + 0$ results in 0, and $0 + 1$ or $1 + 0$ results in 1.

However, when we start doing $1 + 1$, that is technically 2, however we cannot represent 2 since we only have digits 0 and 1. 2 in binary is 10. Therefore, we need to use two digits now, so we end up with 0 then carry 1.

This is quite similar to how we perform addition in the decimal number system. When the sum exceeds the value that can be placed in a single digit, we end up with a carry. This carry is then added to the next higher order of magnitude.



Binary arithmetic

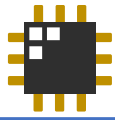
- Convert the two decimal numbers to binary, then add them together

$$\begin{array}{r} 92_{10} \\ + 51_{10} \\ \hline \end{array}$$

4

Let's have quick example showing how adding in binary follows the same concept as adding in decimal.

Here we have two decimal numbers, 92 and 51. Let's add both of them in the binary domain.



Binary arithmetic

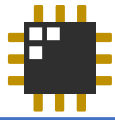
- Convert the two decimal numbers to binary, then add them together

$$\begin{array}{r} 92_{10} \\ + 51_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 1011100_2 \\ + 0110011_2 \\ \hline \end{array}$$

5

92, in binary, will be written as 1011100. I won't be hand-holding the decimal to binary conversion here anymore, but you can do that on your own as an exercise. 51, in binary, will be written as 0110011. Note that I put an extra zero on the left side. I just did this to align the bits properly. Note that adding zero on the left-hand side really does not change the value of the binary number, since you are just technically multiplying zeros with the higher powers of 2.



Binary arithmetic

- Convert the two decimal numbers to binary, then add them together

$$\begin{array}{r} 1 \\ + 92_{10} \\ + 51_{10} \\ \hline 143_{10} \end{array}$$

$$\begin{array}{r} 111 \\ + 1011100_2 \\ + 0110011_2 \\ \hline 10001111_2 \end{array}$$

These two numbers should be equivalent.
Math works!

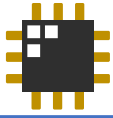
6

Now, let's perform the addition. Starting from the rightmost side, similar to how we do it in decimal, we perform the addition. Remember:

- 0 + 0 results in 0
- 0 + 1 or 1 + 0 results in 1
- 1 + 1 results in 10 (which is 0 carry 1). This is because 1 + 1 equals 2, and in binary, 2 is represented as 10.

We will note that there will be a carry being propagated from right to left, depending on the result of the operation, just like in decimal. Just continue doing the addition until we reach the leftmost bit then we stop.

Now, converting backwards, 10001111 is actually 143 in decimal. You can then verify that in the decimal domain $92 + 51$ is indeed 143. Therefore the math actually works out well!



Binary arithmetic

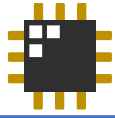
- Addition in binary is simple!
- But how do we subtract?
 - **$X - Y = X + (-Y)$**
 - How do we represent negative numbers?

7

Now, we just concluded that adding in binary is straightforward, we just limit ourselves with 0s and 1s.

But, how do we subtract then?

Subtraction can be thought of as addition with a negative number. So now, the question is how do we represent negative numbers?



Two's complement

- For signed binary numbers, we can use the most significant bit (MSB) (AKA the leftmost bit) to carry the sign information
 - If MSB = 0, the binary number is **positive**
 - If MSB = 1, the binary number is **negative**
- Two's complement is a common method of representing signed binary numbers, we follow the steps below to convert a number to signed:
 1. Start with the absolute value of the number, make sure MSB is 0 (to indicate positive)
 2. Invert all the bits (all 0s -> 1s, all 1s -> 0s), then add 1. Ignore any extra bits.

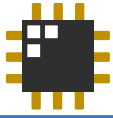
8

To represent negative binary numbers, we use the two's complement notation. Under this notation, the leftmost bit (also called the most significant bit) basically determines whether a number is negative or positive. If the MSB is 0, the number being represented is positive. If the MSB is 1, the number being represented is negative.

There is a specific process that we have to do to convert an unsigned binary representation into the corresponding two's complement notation when representing negative numbers.

We first start by taking the absolute value first, basically the positive equivalent of the number, then write it out in its proper binary representation. Here, we have to make sure that the MSB is actually 0, to indicate that it is a positive number. Again note that putting a 0 on the leftmost bit does not really change the value, so that is fine. Then, we invert all the bits that we have written and then add 1.

This is better demonstrated with an example.



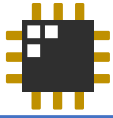
Two's complement

- Write -143 from decimal to binary using two's complement notation

-143₁₀

9

Let's try to convert -143 into binary using two's complement notation.



Two's complement

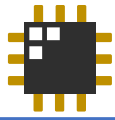
- Step 1: Write the absolute value in binary, ensuring MSB = 0

$$143_{10} = 010001111_2$$

Note: Adding a 0 on the leftmost bit does not change the value!

10

So the first step is to write positive 143 first in binary. So 143 in binary is 10001111. However, we have to add an extra 0 on the MSB to ensure that it represents a positive number under the two's complement notation.



Two's complement

- Step 2: **Invert all bits** then add **1**, ignoring any extra bits.

$$\begin{array}{r} 143_{10} = 010001111_2 \\ 101110000_2 \\ + \quad \quad \quad 1_2 \\ \hline -143_{10} = 101110001_2 \end{array}$$

Note: Adding a 0 on the leftmost bit does not change the value!

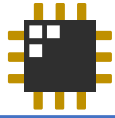
Note: The 1 in the MSB indicates that this is a negative number under two's complement notation

11

Next, we perform the invert everything + 1 procedure. We just do exactly that.

Note that the final result after the addition of 1 should give us a number with an MSB of 1. Under the two's complement notation, we know that an MSB of 1 indicates a negative number now.

Therefore, we can say that under this notation, -143 is equivalent to **101110001** in binary.



Two's complement

- Write -143 from decimal to binary using two's complement notation

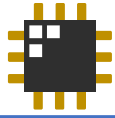
$$\mathbf{-143_{10} = 101110001_2}$$

- Under two's complement notation, the MSB would indicate whether the binary number is positive (0) or negative (1).
 - If MSB = 1, don't translate to decimal right away! Get the absolute value first.
 - We'll *always* state whether the number is signed/written in two's complement

12

It is helpful to note that the MSB of a two's complement number only represents the sign. You should not attempt to convert every bit into decimal right away if we are using two's complement.

Note that we'll always indicate whether a binary number is written in two's complement notation so you will know whether the MSB can be treated as a sign or just a usual binary bit that has a corresponding power of 2 weight.



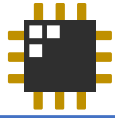
Two's complement

- Convert 101110001 (in two's complement) to its decimal equivalent

101110001₂

13

Let's try to do the reverse. Given this binary number written in two's complement, what is the equivalent decimal number?



Two's complement

- Convert 101110001 (in two's complement) to its decimal equivalent

101110001_2

- This is written in two's complement (as stated), and the MSB is 1 so this is a negative number. What negative number?

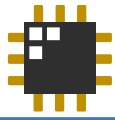
Note: Doing the two's complement 'process' again gives you back the absolute value.

14

Note that the problem stated that the binary number is written in two's complement. Therefore, we check the MSB to check for the sign. Since the MSB is 1, then this implies a negative number.

But that's the only information we have right now. This is a negative number, but what negative number?

We note that the two's complement procedure of inverting everything + 1 can actually be applied once again to get the absolute value of the number.



Two's complement

- **Invert all bits** then add **1**, ignoring any extra bits.

$$\begin{array}{r} 101110001_2 \\ 010001110_2 \\ + \quad \quad \quad 1_2 \\ \hline 143_{10} = 010001111_2 \end{array}$$

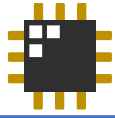
Caution: This process only gives you the absolute value!

Note that the original representation has MSB = 1 (so it is actually negative)

$$\boxed{-143_{10} = 101110001_2} \quad \text{Final answer}$$

15

And so we perform the invert everything + 1 process and we end up with 010001111. What we just computed is the absolute value of the number we are looking for. 010001111 is 143 in decimal. However, note that the original representation has 1 for its MSB. Therefore, the number is actually negative to start with. Thus, we conclude **101110001**, written in two's complement notation is equivalent to -143 in decimal. That's our final answer.



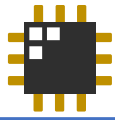
Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -143_{10} \\ + \\ 51_{10} \\ \hline \end{array}$$

16

Let's have another practice. Now, we are trying to add -113 and 21 following the twos complement notation.



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -143_{10} \\ + 51_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 101110001_2 \\ + 000110011_2 \\ \hline \end{array}$$

This is written in two's complement.
This was our answer previously.

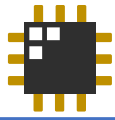
We can pad 0s on the left
to make the #bits equal

17

-143 is the same value that we ended up with in the previous exercise. -143 in two's complement is 101110001.

We also write 51 to its equivalent binary representation. Again, we add extra 0s on the left to align the bits properly. Adding 0s should not change the value.

It's also helpful to indicate that 000110011 is the two's complement representation of 51 already. Note how the MSB is 0 indicating that it is a positive number, thus we don't have to do the 'invert everything + 1' process anymore as this is already positive.



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -143_{10} \\ + 51_{10} \\ \hline \end{array} \quad \begin{array}{r} \textcolor{red}{111} \quad \textcolor{red}{11} \\ 101110001_2 \\ + 000110011_2 \\ \hline \textcolor{red}{110100100}_2 \end{array}$$

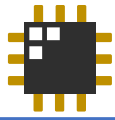
We are using two's complement. The MSB = 1, so this is a negative number. What negative number?

18

Then, we perform the addition. We're already familiar with this process. Take care handling the carry out bits.

We got an answer of 110100100. Since we are working in two's complement, we check the MSB. The MSB here is 1, indicating a negative number. So now, we ask what negative number is this?

So we have to do the 'invert everything + 1' process to find out what the absolute value is.



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -143_{10} \\ + 51_{10} \\ \hline \end{array} \quad \begin{array}{r} 101110001_2 \\ + 000110011_2 \\ \hline 110100100_2 \\ 001011100_2 \end{array}$$

92₁₀ **001011100₂**

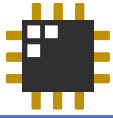
This is the absolute value.
Note that we had MSB = 1.
Therefore, actual answer is negative!

I already did the invert + 1 process
in one go. Verify that this is correct.

19

After doing that 'invert everything + 1' process, we end up with 001011100. This is the absolute value. When converted to decimal, this is 92.

But note that we had an MSB of 1 in the first place before doing this process. Therefore, the actual value should be -92.



Binary subtraction

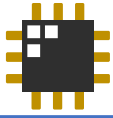
- Perform the following operation using two's complement notation

$$\begin{array}{r} -143_{10} \\ + 51_{10} \\ \hline -92_{10} \end{array} \quad \begin{array}{r} 101110001_2 \\ + 000110011_2 \\ \hline 110100100_2 \end{array}$$

Math works!

20

And there you go. We can also verify that in decimal, $-143 + 51$ is indeed -92 . The math works!



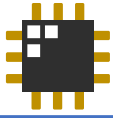
Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 149_{10} \\ - 6_{10} \\ \hline \end{array}$$

21

Let's have another practice. $149 - 6$.



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 149_{10} \\ - 6_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 010010101_2 \\ + 111111010_2 \end{array}$$

Double check these for practice!

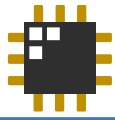
22

We realize that $149-6$ can be expressed as $149 + -6$. Therefore, we just have to add both numbers written in their two's complement forms.

We write 149 in two's complement. It is 010010101. Note that we need to add a 0 on the left to say that it is a positive number under the two's complement notation.

Then, writing -6 in binary, we start with 000000110, then we perform the 'invert everything + 1' process, so we end up with $111111001 + 1 = 111111010$

Now, we are ready to perform the addition.



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 149_{10} \\ - 6_{10} \\ \hline \end{array} \quad \begin{array}{r} 1111 \\ 010010101_2 \\ + 111111010_2 \\ \hline 1010001111_2 \end{array}$$

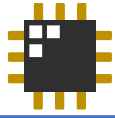
Under two's complement notation, the number of bits should be fixed.
Truncate away any extra carry out bits.

23

Performing the addition, we end up with 1010001111.

Now, note that since we started with 9 bits when we were doing the addition under the two's complement notation. Under the two's complement notation, the result must remain in 9 bits.

Our answer was actually 10 bits long now, since we had an extra carry. Under two's complement notation, the extra bits on the left should be ignored.



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 149_{10} \\ - 6_{10} \\ \hline 143_{10} \end{array} \quad \begin{array}{r} 010010101_2 \\ + 111111010_2 \\ \hline 1010001111_2 \end{array}$$

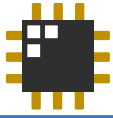
Under two's complement notation, the number of bits should be fixed.
Truncate away any extra carry out bits.

24

Therefore, removing the left most bit to fix the number of bits to 9, which is the number of bits we started with, we will end up with 010001111.

Since the MSB is already 0, this number is already a positive number. We don't have to do the 'invert everything + 1' process anymore, this is the final answer that we need.

Converting 010001111 to decimal, we end up with 143. We can verify that, indeed, $149 - 6$ is equal to 143. The math works out!.



A note on number ranges

- What is the largest (*most **positive***) number in 4-bit two's complement?

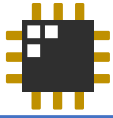
— — — —

25

One thing to note about binary representation is that there is a limit on range of numbers that can be represented given a fixed number of bits.

Let's take for example, 4 bit two's complement notation.

What is the largest positive number that can be represented here?



A note on number ranges

- What is the largest (*most **positive***) number in 4-bit two's complement?

$$\underline{0} \underline{1} \underline{1} \underline{1} = 7_{10}$$

MSB needs to be 0 to represent a positive number (for two's complement)

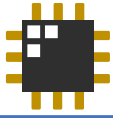
If we want the largest positive number, set the remaining bits to 1

26

As we are following the two's complement notation, the most significant bit (MSB) should be fixed to 0 to indicate a positive number.

Then, we simply max out the magnitude of the number by setting the remaining bits to 1.

Thus, in 4-bits, the most positive number that we can get is 0111 or 7 in decimal.



A note on number ranges

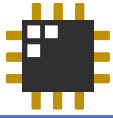
- What is the *most negative* number in 4-bit two's complement?

— — — —

27

Now, since we are working in two's complement notation, negative numbers are supported.

Let's try to determine the most negative number that can be represented in 4-bit two's complement.



A note on number ranges

- What is the *most negative* number in 4-bit two's complement?

$$\underline{1} \underline{0} \underline{0} \underline{0} = -8_{10}$$

MSB needs to be **1** to represent a negative number (for two's complement)

If we want the largest negative number, set the remaining bits to 0

28

Under this notation, the most significant bit should be fixed to 1 to represent a negative number.

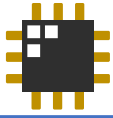
Now, to max out the magnitude for a negative number, we should set the remaining bits to 0.

If we try to get the magnitude of this number, we simply apply the “two's complement process” (invert everything then add 1)

1000 -> 0111 + 1 -> 1000

The magnitude, therefore, is 1000, which is 8 in decimal.

Thus, the most negative number that can be represented is -8 (since we started with a negative representation and just computed for the magnitude later on)



A note on number ranges

- What is the number range representable in **4-bit two's complement**?

$$0111_2 = 7_{10}$$

$$1000_2 = -8_{10}$$

[-8, 7]

Note:

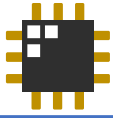
For an *n-bit number*, the two's complement range would be: **$[-(2^{n-1}), 2^{n-1}-1]$**

29

As a summary, here are the values that we have computed.

The range of numbers that can be represented in 4-bit two's complement notation is [-8, 7].

This can be extended to n-bit numbers by following the formula $[-(2^{n-1}), 2^{n-1}-1]$



A note on number ranges

- What is the number range representable in 4-bit unsigned binary?

$$0000_2 = 0_{10}$$

$$1111_2 = 15_{10}$$

[0, 15]

Note:

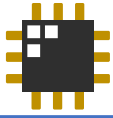
For an *n-bit number*, the unsigned binary range would be: **[0, 2ⁿ-1]**

30

As a quick recap, if we are NOT using two's complement notation (thus, we are using an unsigned binary representation), then we have an effective range of [0, 15].

We simply have to max out the most positive representation by setting all the bits to 1. The smallest number is simply 0, where all bits are set to 0.

This can be extended to n-bit numbers by following the formula [0, 2ⁿ-1]



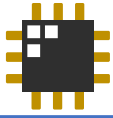
A note on number ranges

- Number ranges matter when doing arithmetic as we can overflow

$$\begin{array}{r} -5_{10} \\ + -5_{10} \\ \hline \end{array}$$

31

The main reason why number ranges matter is that binary arithmetic can overflow. Overflow is a problem when the expected result of the arithmetic exceeds the allowable range of binary numbers that can be represented in a fixed number of bits. Let's take $-5 + -5$ for example.



A note on number ranges

- Number ranges matter when doing arithmetic as we can **overflow**

$$\begin{array}{r} -5_{10} \\ + -5_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 1011_2 \\ + 1011_2 \\ \hline \end{array}$$

Binary arithmetic starts to fail when the expected result is beyond the representable range given a fixed number of bits

$$-10_{10}$$

!=

$$10110_2$$

← This is a positive number under two's complement notation!

Recall:

Under two's complement notation, the number of bits should be fixed.

Truncate away any extra carry out bits.

32

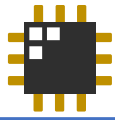
Converting -5 to two's complement notation, we end up with 1011

Adding 1011 and 1011 together, we end up with 10110.

However, recall that under two's complement arithmetic, the number of bits should be fixed. That is, if you used 4 bits for the representation of the operands, the result must also be set to 4 bits.

This leads to $1011 + 1011 = 0110$. This is mathematically wrong, since this implies that the sum of two negative numbers is a positive number (note how the MSB of the result is 0, indicating a positive number)

This is the problem of arithmetic overflow.



A note on number ranges

- Number ranges matter when doing arithmetic as we can **overflow**

$$\begin{array}{r} -5_{10} \\ + -5_{10} \\ \hline -10_{10} \end{array} \quad \begin{array}{r} 11011_2 \\ + 11011_2 \\ \hline 110110_2 \end{array}$$

The solution is to simply **increase** the **number of bits** of the operands so that the number range extends

$-10_{10} = 110110_2$ ← This is a negative number under two's complement notation!

Practice:

Is 10110 really -10 in decimal assuming two's complement notation?

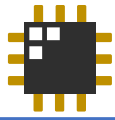
33

The solution here is to simply increase the number of bits of the operands so that the number of bits of the result would be enough to represent the result. Now, with a larger range, the result can be represented correctly.

10110 is a negative number since MSB is 1. We can now perform the “two's complement process” to get the magnitude of the number

10110 $\rightarrow$ 01001 + 1 = 01010

01010 is 10 in decimal. Thus, 10110 (the number that we started with) is -10 in decimal following the two's complement notation.



Arithmetic overflow bugs

- **Arithmetic overflow** can present some problems if left unmanaged
- Let's say a computer uses 8 bits to hold an *unsigned* number

$$\begin{array}{r} 255_{10} \\ + 1_{10} \\ \hline 0_{10} \end{array} = \begin{array}{r} 11111111_2 \\ + 00000001_2 \\ \hline 100000000_2 \end{array}$$

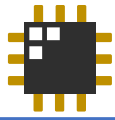
Maximum 8-bit unsigned number

Extra carry bit is truncated away

Note: In computers, data is typically stored in fixed bit-width locations in memory. You can't really change the number of bits during runtime.

Let's say a computer stores numbers using **8 bits** and is working with **unsigned numbers** (meaning no negative values). The highest number that can fit in 8 bits is **255 (11111111 in binary)**. Now, if we try to add **1** to it (**255 + 1**), the result in binary would be **100000000**. But since we only have **8 bits**, the leftmost **1** is **cut off**, and the final value stored becomes **0** instead of **256**.

This happens because computers store numbers in **fixed bit-width**, and they **can't automatically add more bits** when needed. If overflow isn't managed properly, it can lead to wrong results and unexpected errors in programs.



Pac-Man level 256



- The level counter in the original Pac-Man is stored as an **8-bit unsigned integer**. Once you reach level **256**, **arithmetic overflow** occurs.
- Due to how the game draws the icons for the level (fruits), arithmetic overflow **causes it to draw 256 fruits on the screen, overwriting the playing field** with data from other sections in the memory.

Learn more about this here!

https://pacman.fandom.com/wiki/Map_256_Glitch

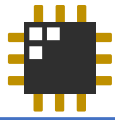
<https://www.youtube.com/watch?v=NKKfW8X9uYk>

35

In the original **Pac-Man**, the game keeps track of levels using an **8-bit number**, which can only count up to **255**. When a player reaches **level 256**, the number goes beyond what the game can handle, causing an **overflow**.

This creates a problem with how the game **displays level icons** (fruits). Normally, each stage is shown with a fruit symbol, but because of the overflow, the game tries to put **256 fruits** on the screen. Since there isn't enough space, these extra icons **replace parts of the game's graphics**, pulling random data from memory. This results in a broken screen where half of the play area becomes messy and nearly impossible to use.

This bug, known as the **"split-screen level"** or **"kill screen"**, is a famous example of old arcade games running into problems because of **limited hardware and programming mistakes**.



Arithmetic overflow bugs

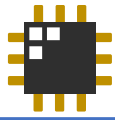
- **Arithmetic overflow** can present some problems if left unmanaged
- Let's say a computer uses 8 bits to hold a *signed* number

$$\begin{array}{rcl} 127_{10} & + & 01111111_2 \leftarrow \text{Maximum 8-bit positive number} \\ + 1_{10} & + & 00000001_2 \\ \hline -128_{10} & = & \underline{10000000}_2 \leftarrow \text{Negative number in two's complement!} \end{array}$$

36

Now, let's say a computer follows the **two's complement system** for signed values. In this system, the largest number that can be stored is **127 (01111111 in binary)**. If we try to add **1** to it (**127 + 1**), the result in binary is **10000000**. However, in **two's complement**, this number actually represents **-128**, not **128**.

This happens because the leftmost bit (called the **sign bit**) determines whether a number is **positive or negative**. When we go past the limit of 127, the sign bit flips, changing the number into a negative value instead of continuing to 128.



Beating Dream Devourer in



Dream Devourer boss in
Chrono Trigger (DS)

Since the Dream Devourer has 32000 HP, which is close to the **signed 16-bit integer** limit, it is actually possible to *defeat it by healing it*.

By using powerful magic that it absorbs during its second phase, it can be (temporarily) healed past **32767*** HP, which overflows the counter and makes its HP **go negative**.

*The maximum positive 16-bit number in two's complement is
 $2^{16-1}-1 = 2^{15}-1 = 32,767$

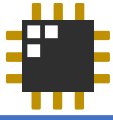
https://chrono.fandom.com/wiki/Dream_Devourer

37

The Dream Devourer has **32,000 HP**, a value dangerously close to the upper limit of a **16-bit signed number**. This creates an unusual exploit where the enemy can be defeated by **healing instead of attacking**.

During its **second phase**, certain spells restore its health instead of dealing damage. If enough healing is applied to push its HP beyond **32,767**, the system experiences an **overflow**, rolling the value into the **negative range**. Since enemies cannot have negative health, the game treats this as a defeat, instantly ending the battle.

This happens because the **16-bit system** can only count up to **32,767** before resetting to **-32,768**, a well-known limitation in older RPGs where unexpected mechanics arise due to numerical constraints.

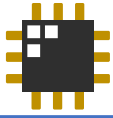


Adders using logic gates

- How to do the addition in hardware?

38

Now, after mastering the addition and subtraction, we start getting curious about how we implement this using logic gates.



Adders using logic gates

- Some observations:

- When adding two binary numbers, we start from right to left.
- We can end up with a *carry out*
- ... which then becomes as a *carry in* for the next column to the left.

$$\begin{array}{r} \text{111} \\ 1011100_2 \\ + 0010101_2 \\ \hline 1110001_2 \end{array}$$

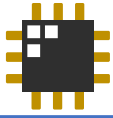
Carry out from the previous columns becoming carry in for the next

We can technically end up adding 3 bits at any given time!

39

Well, if you think about it, the way we add numbers always start from right to left. Then, sometimes we end up with a carry that needs to be propagated to the next column.

So technically, we can end up adding 3 bits at any given time.



Adders using logic gates

- Let's consider only one column at a time

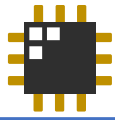
$$\begin{array}{r} C_{out} \quad C_{in} \\ + \quad A \\ \quad B \\ \hline S \end{array}$$

A	B	C _{in}	C _{out}	S
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

40

Therefore, we can actually write a truth table on how we add a single column of bits. We are technically adding just two numbers, but we have to consider the carry in that can be generated from the previous column on the right of it. We also have to consider a carry out that will have to be propagated to the left of the column.

Let's try to complete the truth table!



Adders using logic gates

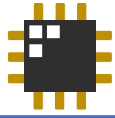
- Let's consider only one column at a time

$$\begin{array}{r} C_{out} \quad C_{in} \\ + \quad A \\ \quad B \\ \hline S \end{array}$$

A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

41

We just have to perform the addition of bits for A, B and C_{in}. Performing the addition and noting what happens to the sum and the generated carry, we end up with the following table.



Adders using logic gates

- We already know what to do from here 😊

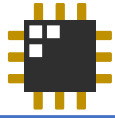
A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

42

So now, given a complete truth table, we already know what we need to do to write a corresponding Boolean expression for this.

If you can recall, we check out for all the rows with an output of 1, then write expressions on a per row basis, using NOT and AND gates depending on the condition, then finally OR all of the terms together to end up with the final expression.

Note that we have two outputs here, S and Cout. Therefore, we should have two Boolean expressions for this table.



Adders using logic gates

- We already know what to do from here 😊

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

Note that the '+' symbol here is the OR logic, not addition

A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

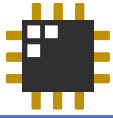
43

Doing the process of writing a Boolean expression from the truth table, we end up with the following.

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

Note that the + symbol here means the logic OR, not the addition symbol.



Adders using logic gates

- If you search online, expressions might be written differently...
 - They're equivalent, don't worry.

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

$$C_{out} = BC_{in} + AC_{in} + AB$$

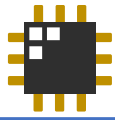
Note that the '+' symbol here is the OR logic, not addition

The $\oplus$ symbol here implies the XOR logic.
XOR: $Y = A \oplus B = A'B + AB'$

44

Note that if you try to search online, you might find a different expression for Sum and Cout. I just like to highlight that they are logically equivalent. There's a new symbol here, this is the XOR logic, which has a corresponding Boolean expression $A'B + AB'$.

On the next slide, we'll try to derive the equivalence between these two expressions.



Adders using logic gates

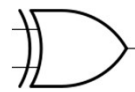
- If you search online, expressions might be written differently...
 - They're equivalent, don't worry.

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

$$S = C_{in}'(A'B + AB') + C_{in}(A'B' + AB)$$

$$S = C_{in}'(A \oplus B) + C_{in}(A \oplus B)'$$

$$S = A \oplus B \oplus C_{in}$$



XOR gate

XOR truth table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

The $\oplus$ symbol here implies the XOR logic.

$$\text{XOR: } Y = A \oplus B = A'B + AB'$$

$$(A \oplus B)' = AB + A'B'$$

45

So for the S expression. We just group the terms in terms of C_{in} for now. Then we end up with:

$$S = C_{in}'(A'B + AB') + C_{in}(A'B' + AB)$$

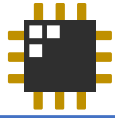
Then, given the XOR logic, which is shown on the right together with its truth table, we identify that the pattern $A'B + AB'$ is actually $A \text{ XOR } B$, while $A'B' + AB$ is actually $(A \text{ XOR } B)'$

Continuing the logic, we end up with $C_{in}'(A \text{ XOR } B) + C_{in}(A \text{ XOR } B)'$.

However, we note that this follows the same pattern as before $(XY' + X'Y)$, which will give us:

$$S = A \text{ XOR } B \text{ XOR } C_{in}.$$

Thus, we prove the logical equivalence.



Adders using logic gates

- If you search online, expressions might be written differently...
 - They're equivalent, don't worry.

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in} + ABC_{in} + ABC_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in} + ABC_{in} + ABC_{in}$$

$$C_{out} = BC_{in}(A' + A) + AC_{in}(B' + B) + AB(C_{in}' + C_{in})$$

$$C_{out} = BC_{in} + AC_{in} + AB$$

This is just a Boolean simplification procedure similar to the earlier lectures!

46

For the Cout term on the other hand, we can just replicate the ABCin term, that will allow us to factor out the following terms:

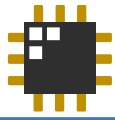
Bcin, Acin and AB.

After factoring out, we will end up with this expression:

$$C_{out} = BC_{in}(A' + A) + AC_{in}(B' + B) + AB(C_{in}' + C_{in})$$

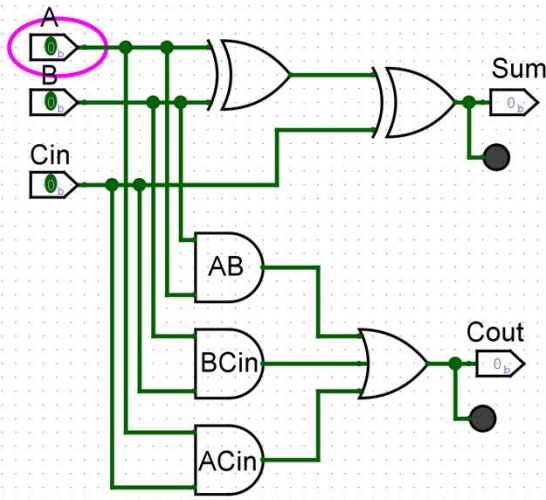
However, we note that the pattern $A + A'$ is actually equivalent to 1 from our Boolean theorems. Moreover, $A(1)$ is just A. Therefore the expression reduces down to:

$$C_{out} = BC_{in} + AC_{in} + AB$$



Adders using logic gates

- Adder (AKA full adder) implementation using logic gates!



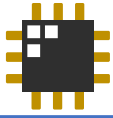
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = BC_{in} + AC_{in} + AB$$

A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

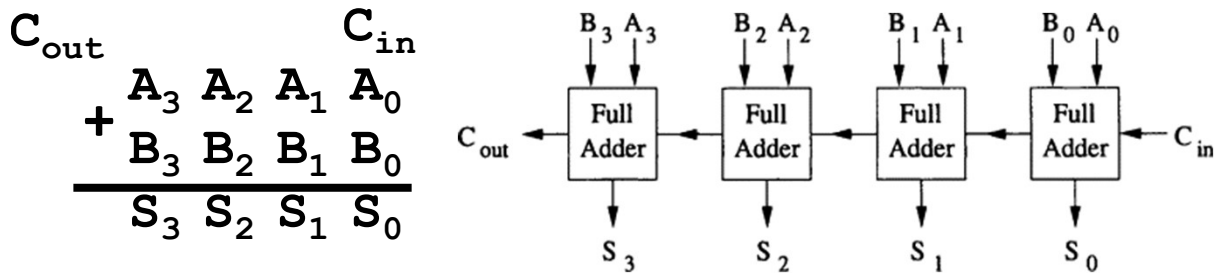
47

Of course, we can write the expressions up and demonstrate that the addition works. Check how A, B, and Cin are triggered in different manner to show all possible combinations, then we can see the values corresponding to the sum and carry out.



Adders using logic gates

- We can use multiple full adders to implement the binary addition!



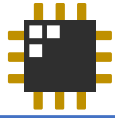
48

So now, how do we add more bits?

Well, it is simply done by chaining the adders that we have implemented before. We note that the carry out of the rightmost bit will become the carry in of the bit on its left and so on.

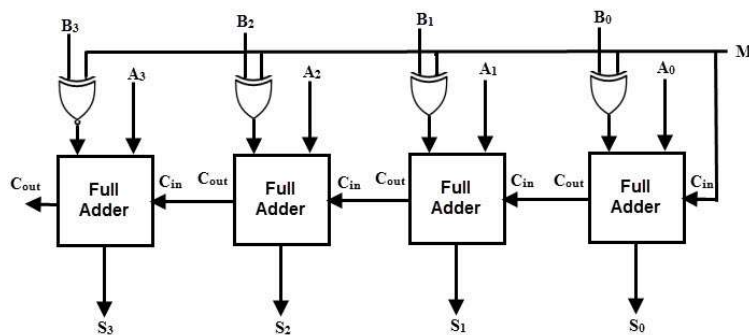
This pattern continues until we reach the leftmost bit. In that case, we will end up with an extra carry out in our output.

Try to check the illustration on the left and the connections done on the illustration on the right to verify how they are doing the multi-bit addition.



Addition and subtraction?

- By leveraging two's complement, we can add and subtract!

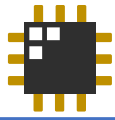


49

One of the great things about **two's complement** is that we do not need a special circuit to subtract numbers. We can do it using the same **adder** that we use for addition.

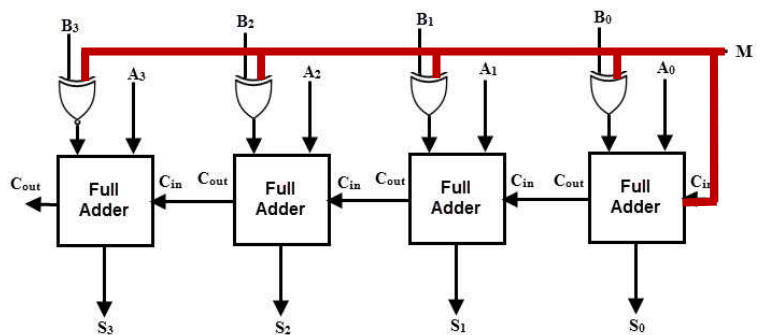
In two's complement, subtracting a number is the same as **adding its negative version**. To find the negative version of a number, we **flip all the bits** and then **add 1**. After doing this, we can just add the result like a normal addition.

This method makes things simpler because the computer only needs **one circuit** for both adding and subtracting. It is a smart way to save space and make calculations faster.



Addition and subtraction?

- What happens if **M = 0**?



$$A_3 A_2 A_1 A_0 + B_3 B_2 B_1 B_0 + 0 = S_3 S_2 S_1 S_0$$

Normal addition process



XOR gate

XOR truth table

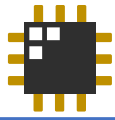
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$0 \oplus B = B$$

50

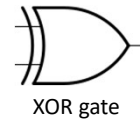
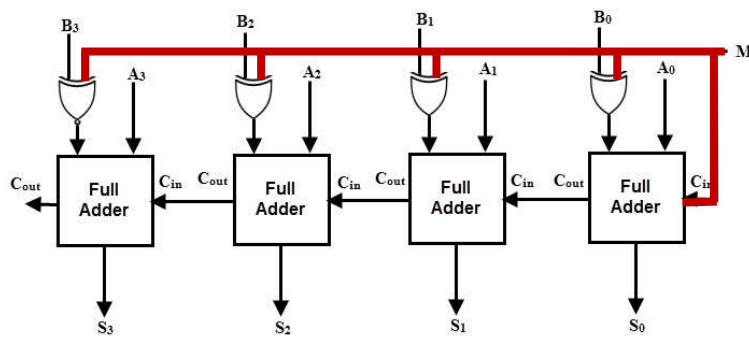
In this example, the input **M = 0** is **XORed** with the **B_n** terms. Based on the truth table, when **M = 0**, the XOR operation just copy the value of **B_n**, since any number **XORed** with **0** remains the same. Additionally, the **carry-in (Cin)** is also **0** because it is connected to **M = 0**.

As a result, the adder performs a simple sum of **A, B (which remains unchanged)**, and the **carry-in (M = 0)**. In other words, the operation being carried out is just **A + B**.



Addition and subtraction?

- What happens if **M = 1**?



XOR truth table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$1 \oplus B = B'$$

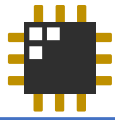
$$A_3 A_2 A_1 A_0 + B_3' B_2' B_1' B_0' + 1 = S_3 S_2 S_1 S_0$$

Invert everything + 1

51

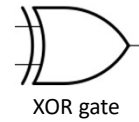
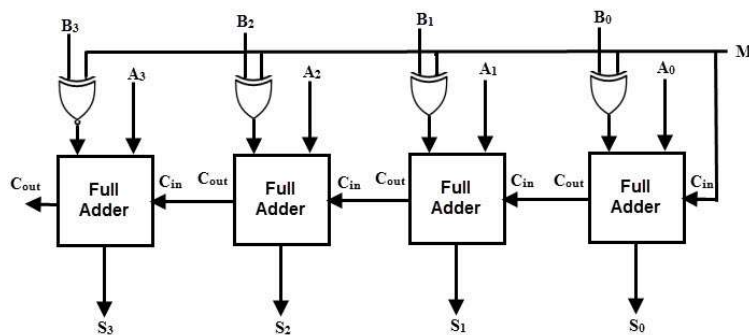
In this example, the input **M = 1** is **XORed** with the **B_n** terms. According to the truth table, when **M = 1**, the XOR operation **inverts** the value of **B_n**, since any number **XORed with 1 becomes its complement**. As a result, all bits of **B** are flipped, giving us **B₃'B₂'B₁'B₀'**. Additionally, the carry-in (**C_{in}**) is set to **1** because **M = 1**. This process is exactly how we perform **two's complement**, which is used to represent negative numbers in binary.

So, in this case, the adder is actually summing **A**, the **inverted version of B**, and the **carry-in (M = 1)**, effectively computing **A - B** instead of just addition.



Addition and subtraction?

- We can use an XOR gate as a conditional inverter!



XOR gate

XOR truth table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

- If $M = 0$ (addition), addition proceeds normally, $C_{in} = 0$
- If $M = 1$ (subtraction), we invert $B_3 B_2 B_1 B_0$ and $C_{in} = 1$

52

As a final note, since we did a subtraction process before, we can also perform subtraction using logic gates.

The XOR logic allows for a conditional inverter implementation. As you can see in the truth table on the right, if the B input is held at 1, then the output Y is just the inverse of A. However, if the B input is held at 0, then the output Y is just exactly A.

Therefore, if the other input of the XOR is 1, we are performing inversion, but if it is 0, then we are not inverting.

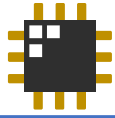
Thus, we can write them up like this. Here, the B_3, B_2, B_1, B_0 inputs are being connected to the XOR gates. This means, that if the input M is 1, they are being inverted.

Moreover, as you can see, M is also connected to the C_{in} of the rightmost bit.

Therefore, if $M = 1$, we are actually inverting all the bits of B and adding 1. This process sounds familiar right? This is the two's complement procedure we highlighted earlier.

Thus, we can conclude that if M is 1, we are actually doing $A - B$. But if M is 0, then we are doing $A + B$.

That is how addition and subtraction is being implemented using logic gates. 😊



Designing magnitude comparators

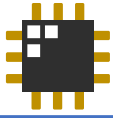
- We can leverage a subtractor to implement **comparators**
- **Comparators** indicate whether one input is greater than (**G**), less than (**L**), or equal (**E**) to a second input.
- **Practice: Design a magnitude comparator with two inputs C and D**
 - Assume inputs are always positive (MSB = 0)

53

Now, let's utilize the circuit that we have on previous example for other purposes. In this case, **subtractor** can also work as a **comparator** by checking the result of subtraction. Instead of just finding the difference, we use it to see if one number is **bigger, smaller, or the same** as another.

A **magnitude comparator** does this by looking at the subtraction output. If the result is **zero**, the numbers are **equal**. If the result is **positive**, the first number is **greater**. If it is **negative**, the second number is **greater**.

Let's practice and design a magnitude comparator with two inputs C and D, and let's assume that inputs are always positive.



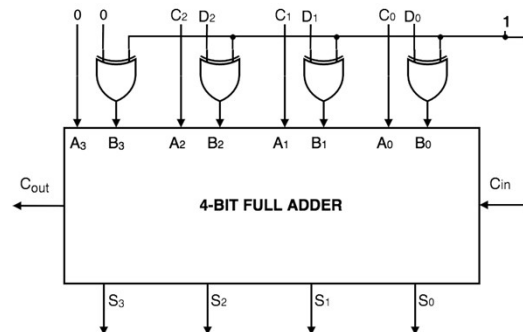
Designing magnitude comparators

- **Practice: Design a magnitude comparator with two inputs C and D**
 - Assume inputs are always positive (MSB = 0)

- **Idea:**

- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less than** D
- If $C - D > 0$, C is **greater than** D

**Utilize a subtractor to
do this comparison!**



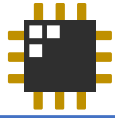
54

Our goal is to compare two **4-bit inputs, C and D**, and determine their relationship: **C > D or C < D, or C = D**.

To do this, we use a **subtractor circuit** (with **C_{in} = 1**, as shown earlier) that takes **C and D** as inputs. The circuit produces a **3-bit output**, where each bit represents a specific comparison result:

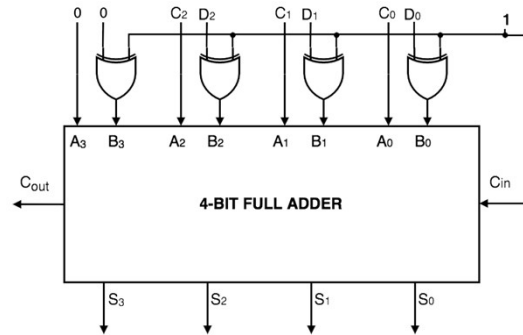
- **G (greater than):** Set to **1** if **C is larger than D**
- **L (less than):** Set to **1** if **C is smaller than D**
- **E (equal to):** Set to **1** if **C and D are the same**

This setup allows the circuit to clearly indicate how the two numbers compare.



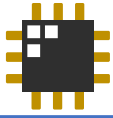
Designing magnitude comparators

- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less than** D
- If $C - D > 0$, C is **greater than** D



55

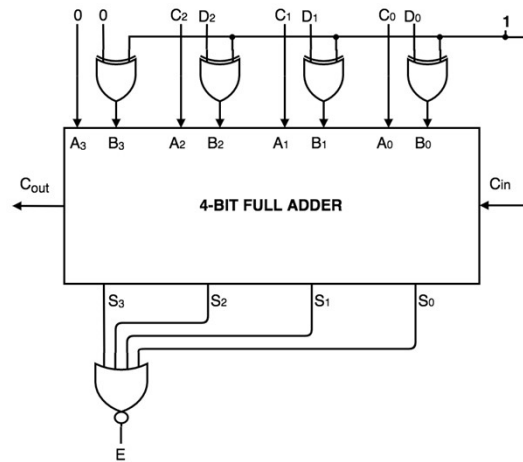
Here, we can determine that **C and D are equal** if their difference ($C - D$) results in **0**. Using this idea, we can create a logical expression that ensures the **E (equal) output is 1** whenever the subtraction result is zero.



Designing magnitude comparators

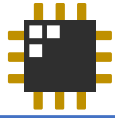
- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less** than D
- If $C - D > 0$, C is **greater** than D

$$E = (S_3 + S_2 + S_1 + S_0)'$$



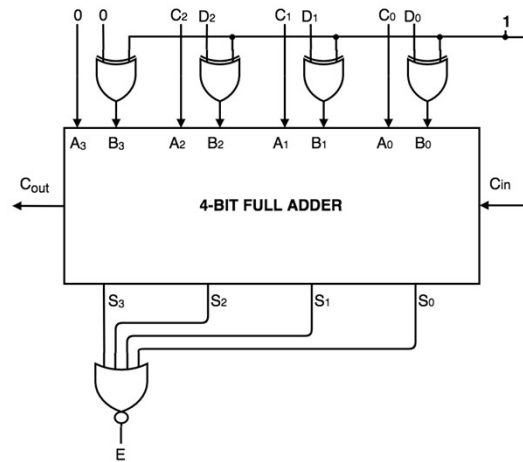
56

Now, how do we check if the output is **0**? Every bit of **S** must also be **0**. One way to express this is **S3' S2' S1' S0'**, where each bit is inverted and then ANDed together. However, we can simplify this using **DeMorgan's Theorem**. Instead of applying a **NOT** gate to each bit and then **ANDing** them, we can take advantage of the **NOR gate**, which directly gives the desired result in a more efficient way. This reduces the number of logic gates needed, making the circuit simpler.



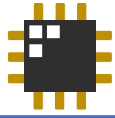
Designing magnitude comparators

- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less than** D
- If $C - D > 0$, C is **greater than** D



57

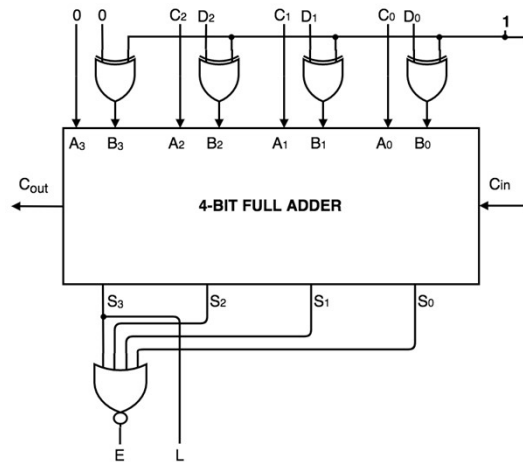
If the result of $C - D$ is negative, we can conclude that **C is less than D**. We can use this logic to derive the logical expression for the **L (less than) output to be 1**.



Designing magnitude comparators

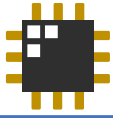
- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less than** D
- If $C - D > 0$, C is **greater than** D

$$L = S_3$$



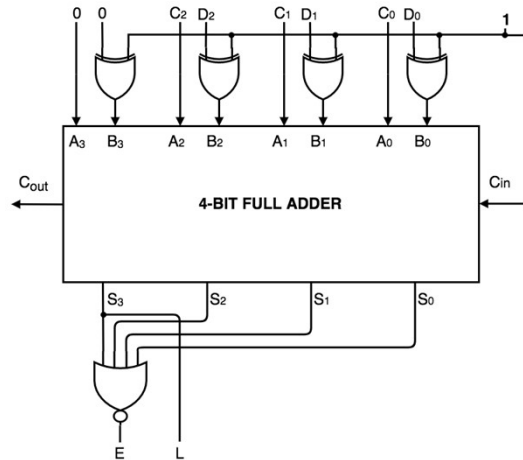
58

How do we know if the output is **less than 0**? In **two's complement notation**, a negative number is indicated when the **most significant bit (MSB)** is 1. Since our result follows this format, we can directly use **S3** to represent the **L (less than) output**, meaning **L = S3**.



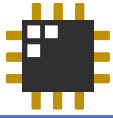
Designing magnitude comparators

- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less than** D
- If $C - D > 0$, C is **greater than** D



59

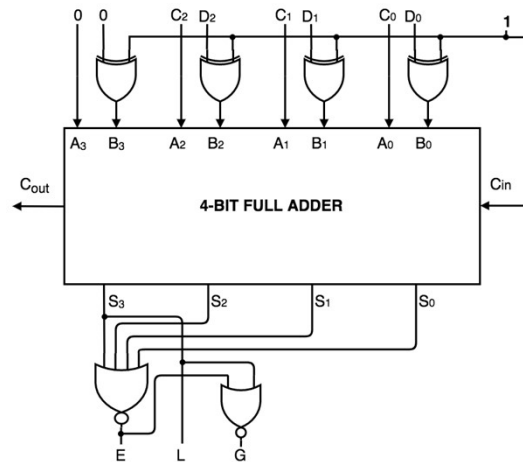
To determine if **C is greater than D**, we need the result of **C - D** to be **positive**. This is the opposite of the "less than" condition and confirms that **C is larger than D**.



Designing magnitude comparators

- If $C - D = 0$, C is **equal** to D
- If $C - D < 0$, C is **less than** D
- If $C - D > 0$, C is **greater than** D

$$G = (E + L)'$$



60

Lastly, to determine if the output is **greater than 0**, we check if it is **neither equal to zero nor negative**. This means **C is greater than D** when **E'L'** holds true.

Again, using **DeMorgan's Theorem**, we can simplify this expression by extracting the complement and inverting the AND operation, resulting in:

$$G = (E + L)'$$